Project Proposal & Development Plan

Machine Learning Operations

Basit, Lepori, Pierro, Tomasi, Ziurkalov

February 12^{th} 2026

Contents

1 Project Summary **2**
- 1.1 Scope and Objectives . . . 2
- 1.2 Relevance . . . 2

2 Deliverables **3**
- 2.1 Deliverables Overview . . . 3
- 2.2 D1. Data Pipeline . . . 3
- 2.3 D2. ML Kernel . . . 3
- 2.4 D3. CI/CD and Reproducible Execution . . . 4
- 2.5 D4. Monitoring and Dashboarding . . . 5

3 Milestones **6**

4 Work Breakdown Structure (WBS) **8**

5 Sprint Plan **10**

6 Definition of Done and Definition of Ready **12**
- 6.1 Definition of Done (DoD) . . . 12
- 6.2 Definition of Ready (DoR) . . . 12

7 Resources & Infrastructure **14**
- 7.1 Technology Stack . . . 14
- 7.2 Execution Environment . . . 14

Project Summary

Scope and Objectives

This project focuses on the analysis and optimization of patient flows within a hospital Emergency Department (ED) through the integration of Process Mining techniques and Machine Learning Operations (MLOps) practices.

The primary objectives are:

- to reconstruct and analyze real patient pathways from event log data,
- to identify operational bottlenecks and inefficiencies affecting patient waiting times,
- to develop predictive models for key operational indicators such as waiting time and next activity occurrence,
- to ensure full reproducibility, traceability, and portability of the analytical pipeline through MLOps principles.

The system is designed as a batch-oriented analytical framework and explicitly excludes real-time clinical decision-making or direct interaction with hospital information systems.

Relevance

Emergency Departments operate under high uncertainty and resource constraints, where inefficient patient flow management can lead to overcrowding, prolonged waiting times, and increased clinical risk.

By transitioning from static, retrospective analysis to a reproducible and extensible data-driven system, this project enables continuous operational insight generation. The proposed architecture supports future extensions toward real-time monitoring, resource-aware analysis, and adaptive learning under changing hospital conditions.

Deliverables

The project delivers a complete and modular MLOps system designed to analyze, model, and monitor patient flows in a hospital Emergency Department. Deliverables are structured to explicitly cover the four required dimensions: **Data Pipeline**, **ML Kernel**, **CI/CD**, and **Monitoring**, while supporting reproducibility, extensibility, and operational transparency.

Deliverables Overview

Deliverables are organized as interconnected components forming an end-to-end analytical workflow. Each deliverable corresponds to a clearly defined responsibility within the system architecture and can be validated independently.

D1. Data Pipeline

The Data Pipeline implements the ingestion, validation, and transformation of raw Emergency Department event logs into a structured and analysis-ready format.

Core functionalities include:

- schema validation and consistency checks to ensure data integrity,
- robust timestamp parsing, normalization, and timezone handling,
- strict ordering of events within patient encounters,
- export to a canonical event log representation compatible with process mining and machine learning components.

The pipeline is deterministic and fully reproducible when executed within the Docker environment, enabling consistent results across executions and team members. Versioned datasets and intermediate outputs support traceability and controlled experimentation.

D2. ML Kernel

The Machine Learning Kernel represents the core analytical intelligence of the system. It transforms engineered event-log features into predictive signals supporting operational analysis of Emergency Department processes.

Prediction Tasks

Given the sequential nature of event log data, the kernel supports two complementary prediction tasks:

- **T1. Waiting Time Regression:** estimation of the waiting time (in minutes) between two consecutive activities within the same patient encounter.
- **T2. Next Activity Classification:** prediction of the next activity in the patient pathway given the current activity.

Feature Engineering

Features are computed at the transition level and capture temporal, process-related, and historical context while avoiding information leakage:

- temporal context (hour of day, day/night, weekday/weekend, holiday indicators),
- process context (current activity label, emergency flag),
- historical context (cumulative waiting time, maximum observed wait, number of completed events).

Training and Evaluation

Model training is executed in batch mode on historical data and follows a reproducible workflow:

- encounter-level data splitting to prevent leakage across patient traces,
- baseline models (e.g., Random Forest) for reference and robustness,
- gradient-boosted tree models (XGBoost) for primary regression and classification tasks,
- fixed random seeds, containerized execution, and versioned artifacts.

Evaluation is performed using task-specific metrics:

- MAE and RMSE for waiting time regression, including tail-focused analysis,
- accuracy and macro-averaged F1-score for next-activity classification.

All trained models, encoders, and evaluation summaries are persisted as artifacts, ensuring auditability and reproducible inference.

D3. CI/CD and Reproducible Execution

Continuous Integration and Continuous Deployment (CI/CD) capabilities are addressed through reproducible execution and automated validation mechanisms.

At the current stage, the system provides:

- containerized execution via Docker to guarantee environment consistency,
- deterministic pipelines validated through scripted entry points,
- compatibility with lightweight CI workflows for code quality checks, dependency validation, and pipeline execution testing.

While full automated deployment is not implemented, the architecture is explicitly designed to support future CI/CD extensions, including automated retraining, artifact validation, and deployment orchestration.

D4. Monitoring and Dashboarding

Monitoring capabilities focus on providing transparency and interpretability over both process behavior and model outputs.

An interactive Streamlit dashboard enables:

- visualization of discovered process models (frequency and performance views),
- exploration of waiting time distributions across temporal slices (day/night, weekday/weekend, holidays),
- inspection of model predictions and evaluation summaries,
- identification of transitions associated with elevated delay risk.

Monitoring is designed as a post-processing and analysis layer, supporting operational insight and model performance assessment rather than real-time clinical intervention.

Milestones

To ensure effective monitoring of project progress, a defined set of key milestones has been established. Each milestone represents a verifiable project outcome, marking the formal completion of a major development phase or deliverable. This milestone-based framework provides clear and objective checkpoints for tracking schedule adherence, managing resources, and supporting informed governance decisions throughout the project lifecycle.

- **M1: Project Definition and Planning.**
  The overall project framework is formally defined and approved. This milestone includes the consolidation of project objectives, technical scope, development methodology, and collaboration workflow. The System Specification Document and initial project plan are reviewed and validated, establishing a common understanding across the team.

- **M2: Data Pipeline Implementation and Validation.**
  The data ingestion and preprocessing pipeline is fully implemented. Raw Emergency Department event logs are cleaned, validated, and transformed into a standardized and versioned dataset suitable for downstream process mining and machine learning tasks. Reproducibility is verified through deterministic execution within the containerized environment.

- **M3: Process Mining Analysis and Reporting.**
  Process discovery and performance analysis are completed using the processed event log. Frequency-based and performance-based process models are generated and validated, enabling the identification of dominant patient pathways, bottlenecks, and critical waiting time transitions. Analytical reports and visual artifacts are finalized.

- **M4: Machine Learning Kernel Development.**
  Feature engineering and model training pipelines are implemented and validated. Baseline and candidate machine learning models are trained to predict waiting times and next activity occurrences. Evaluation metrics are computed and reviewed, and model artifacts are persisted, marking the functional completion of the ML Kernel.

- **M5: Model Freeze and Reproducibility Verification.**
  The predictive models and feature extraction logic are frozen. End-to-end reproducibility of training, evaluation, and inference is verified through Docker execution, fixed random seeds, and versioned artifacts. This milestone establishes a stable analytical baseline for deployment and further extensions.

- **M6: Dashboard Integration and Monitoring Capabilities.**
  The interactive Streamlit dashboard is fully integrated with process mining outputs and model predictions. Users can explore process maps, waiting time distributions, and predictive results. Monitoring focuses on post-hoc analysis and performance inspection, providing transparency over system behavior and model outputs.

- **M7: Containerized System Release.**
  The complete end-to-end system is packaged into a production-ready Docker image. All components—data pipeline, ML kernel, and dashboard—are runnable through

a unified execution interface, ensuring portability and consistent behavior across environments.

- **M8: Future Architecture Definition and Project Closure.**
  The project concludes with the consolidation of results, final evaluation, and completion of all required documentation. A future architecture is defined, outlining extensions such as resource-aware process mining, CI/CD automation, and live data integration, marking formal project closure.

Work Breakdown Structure (WBS)

To organize the project effectively, a Work Breakdown Structure (WBS) has been defined. The WBS decomposes the project into clear, manageable phases, tasks, and subtasks, providing a structured foundation for planning, scheduling, responsibility assignment, and progress tracking throughout the development lifecycle.

Each work package corresponds to a major functional area of the system and is aligned with the defined milestones and deliverables.

WP1: Project Initiation and Data Engineering

This work package focuses on establishing the project foundation and preparing the data infrastructure required for all downstream activities.

- **WP1.1 Project initiation and planning**
  - definition of project objectives and scope,
  - agreement on development methodology and collaboration workflow.
- **WP1.2 Event log ingestion and validation**
  - raw Emergency Department event log ingestion,
  - schema validation and consistency checks,
  - timestamp parsing, normalization, and ordering of events within encounters.
- **WP1.3 Dataset versioning and reproducibility**
  - configuration of dataset versioning mechanisms,
  - validation of deterministic and reproducible data pipeline execution.

WP2: Process Mining and Operational Analysis

This work package addresses the discovery and analysis of patient flows through process mining techniques.

- **WP2.1 Process discovery**
  - generation of frequency-based Directly-Follows Graphs,
  - identification of dominant patient pathways.
- **WP2.2 Performance analysis**
  - computation of waiting times between consecutive activities,
  - generation of performance-based process models.
- **WP2.3 Bottleneck identification and reporting**
  - detection of critical transitions and delays,
  - production of analytical reports and visual artifacts.

WP3: Machine Learning Development

This work package covers feature engineering, predictive modeling, and evaluation activities forming the ML Kernel of the system.

- **WP3.1 Feature engineering**
  - extraction of temporal, process, and historical features,
  - prevention of information leakage across patient traces.
- **WP3.2 Model training**
  - implementation of baseline and candidate models,
  - training of regression and classification models in batch mode.
- **WP3.3 Model evaluation and validation**
  - computation of task-specific performance metrics,
  - analysis of model behavior under critical conditions.

WP4: Operations, Deployment, and Monitoring

This work package focuses on system integration, deployment readiness, and monitoring capabilities.

- **WP4.1 Dashboard development**
  - implementation of the Streamlit-based interactive dashboard,
  - integration of process mining outputs and model predictions.
- **WP4.2 Containerization and execution**
  - Docker-based packaging of the full pipeline,
  - validation of end-to-end reproducible execution.
- **WP4.3 Monitoring and inspection**
  - post-hoc monitoring of process behavior and model outputs,
  - performance inspection across temporal and operational slices.

WP5: Extensions and Future Work

This work package defines planned extensions beyond the core project scope, providing a roadmap for future evolution.

- **WP5.1 Resource-aware process mining**
  - integration of staff and resource information,
  - analysis of workload distribution and resource utilization.
- **WP5.2 Live data integration design**
  - architectural design for streaming event ingestion,
  - evaluation of real-time prediction and monitoring feasibility.

Sprint Plan

The project follows an Agile development approach based on time-boxed sprints. This iterative structure supports incremental delivery, continuous assessment, and controlled refinement of system components. The sprint plan maps the Work Breakdown Structure (WBS) to a sequence of sprints, reflecting the actual project progress up to mid-February.

Sprint Schedule and Scope

Sprint ID	**Start Date**	**Week(s)**	**Primary Focus (WBS Alignment)**
Sprint 0	22/12/2025	Week 0	Project ideation, scope definition, and planning (WP1.1)
Sprint 1	05/01/2026	Week 1–2	Data ingestion, validation, and preprocessing pipeline implementation (WP1.2)
Sprint 2	19/01/2026	Week 3–4	Dataset versioning, reproducibility setup, and process mining analysis (WP1.3, WP2)
Sprint 3	02/02/2026	Week 5	Feature engineering and machine learning model development (WP3.1, WP3.2)
Sprint 4	09/02/2026	Week 6	Model evaluation, dashboard integration, and monitoring setup (WP3.3, WP4.1, WP4.3)
Sprint 5	10/02/2026	Week 6–7	Dashboard refinement, usability improvements, and documentation finalization (WP4.1, WP4.3)
Sprint 6	17/02/2026	Week 8	Future architecture definition, monitoring consolidation, and project closure (WP5)

Table 1: Sprint Plan and WBS Alignment

Sprint Governance and Coordination

Sprint execution is governed by a lightweight coordination framework inspired by Agile best practices. Each sprint includes a planning phase to define objectives and acceptance criteria, followed by implementation and a final review to validate deliverables.

Coordination activities include:

- regular planning and review meetings aligned with sprint boundaries,
- continuous asynchronous coordination through shared communication channels,
- task tracking aligned with the Work Breakdown Structure to ensure traceability between sprints, deliverables, and milestones.

Project Phasing

The sprint sequence implements a clear project phasing strategy:

- **Sprint 0 (Foundation):** project definition and planning.
- **Sprints 1–2 (Data and Process Analysis):** data pipeline stabilization, reproducibility setup, and process mining.
- **Sprints 3–4 (Model Development and Integration):** feature engineering, predictive modeling, evaluation, and initial dashboard integration.
- **Sprint 5 (Refinement):** dashboard improvements, monitoring consolidation, and usability enhancements.
- **Sprint 6 (Closure and Future Outlook):** documentation completion and definition of future extensions, including additional features and architectural enhancements.

This phased approach reflects the current project status, where core deliverables are largely completed and remaining effort focuses on consolidation, refinement, and future-oriented planning rather than new core functionality.

Definition of Done and Definition of Ready

Clear and objective completion and readiness criteria are essential to ensure consistent development quality, predictable progress, and effective coordination across all project activities. This project adopts formal **Definition of Done (DoD)** and **Definition of Ready (DoR)** frameworks to establish measurable conditions for initiating and closing development tasks throughout the MLOps lifecycle.

Definition of Done (DoD)

The Definition of Done specifies the conditions under which a work item, deliverable, or milestone is considered fully completed. It serves as a quality gate for sprint reviews and milestone validation, ensuring that outputs are not only implemented but also validated, reproducible, and documented.

Criterion	Requirements
Implementation Complete	Development completed in accordance with the System Specification Document and approved architectural design.
Requirements Satisfied	All applicable functional and non-functional requirements related to the deliverable are fully met.
Testing and Validation	Relevant validation steps completed (e.g., pipeline execution, model evaluation, process mining verification) with no critical defects.
Data Flow Operation	All data pipelines, feature extraction steps, and model training or inference processes execute correctly under expected conditions.
Reproducibility Guaranteed	Execution is fully reproducible via Docker, fixed random seeds, and versioned artifacts (data, models, configurations).
Documentation Updated	All relevant technical documentation, including usage instructions and configuration details, is completed or updated.
Formal Acceptance	Deliverable reviewed and formally accepted during sprint review or milestone evaluation.

Table 2: Definition of Done (DoD) Criteria

The Definition of Done ensures that completed work is functionally correct, validated, documented, and ready for integration within the broader MLOps pipeline.

Definition of Ready (DoR)

The Definition of Ready defines the conditions that must be satisfied before a task or work item can enter the development phase. This framework ensures that implementation begins only when sufficient clarity, resources, and dependencies are in place, reducing uncertainty and rework.

Criterion	**Requirements**
Objective and Scope Defined	Clear and concise definition of the task objective, aligned with project goals and milestones.
WBS Alignment	Task explicitly linked to a Work Breakdown Structure element and assigned to a specific sprint with a defined timeline.
Requirements Clarity	All functional and non-functional requirements identified, with no unresolved ambiguities or contradictions.
Data and Resources Available	Required datasets, scripts, computing resources, and software dependencies are available and accessible.
Dependencies Resolved	All prerequisite tasks, milestones, or external dependencies are completed or formally accepted.
Acceptance Criteria Defined	Measurable and verifiable acceptance criteria specified for objective evaluation of completion.
Technical Feasibility	Technical feasibility assessed with reasonable effort and duration estimates, and no blocking technical constraints identified.

Table 3: Definition of Ready (DoR) Criteria

By enforcing these readiness conditions, the project ensures that development activities begin in a controlled and well-defined context, supporting predictable sprint execution and effective resource utilization.

Resources & Infrastructure

The successful implementation of the proposed system relies on a well-defined set of software tools, execution environments, and infrastructure resources. All selected technologies are aligned with the project requirements in terms of reproducibility, portability, scalability, and ease of maintenance, which are central principles in an MLOps-oriented development lifecycle.

Technology Stack

The system is primarily developed in Python, chosen for its extensive ecosystem supporting data engineering, process mining, machine learning, and visualization tasks.

- **Programming Language:** Python 3.9 or later is used as the main development language, ensuring compatibility with modern data science and MLOps libraries.
- **Process Mining Framework:** PM4Py is employed for event log handling, process discovery, and performance analysis. Its compliance with the XES standard and rich analytical capabilities make it suitable for modeling and analyzing Emergency Department workflows.
- **Machine Learning Libraries:** Scikit-learn provides a stable and well-tested foundation for baseline models, preprocessing, and evaluation routines. XGBoost is used for advanced tree-based models due to its strong performance on tabular data, robustness to heterogeneous feature types, and support for reproducibility through deterministic training.
- **MLOps and Versioning Tools:** Docker is used to containerize the entire pipeline, guaranteeing environment consistency across development and execution platforms. DVC (Data Version Control) is employed to track datasets, intermediate artifacts, and model outputs, while Git is used for source code versioning and team collaboration.
- **Visualization and Reporting:** Streamlit is adopted to build an interactive dashboard for exploratory analysis and result presentation. Matplotlib supports the generation of static plots used in reporting and performance analysis.

Execution Environment

The project is designed to run in a controlled and reproducible execution environment, with a primary focus on batch processing of historical event log data.

- **Development Environment:** Local development environments are configured using explicit dependency specification files (e.g., `requirements.txt`), ensuring consistency across team members and minimizing environment-related issues.
- **Compute Resources:** All computational workloads are executed on CPU-based environments, which are sufficient for process mining and tabular machine learning tasks. The system does not require specialized hardware such as GPUs.

- **Storage and Artifact Management:** Input datasets, intermediate results, and trained model artifacts are stored either locally or in remote storage backends integrated with DVC. This setup enables dataset versioning, experiment traceability, and reproducible re-execution of the entire pipeline.
- **Containerized Execution:** The Docker-based execution environment encapsulates all dependencies and runtime configurations, allowing the system to be executed consistently across different machines and operating systems without manual setup.

Overall, the selected resources and infrastructure provide a balanced trade-off between technical robustness and implementation simplicity, while leaving room for future extensions such as cloud deployment, automated CI/CD pipelines, and real-time data integration.